Authorability Without Prior: Measuring Whether Structured Tooling Replaces Training Data for a New Programming Language

Eduardo Díaz
ediaz@dcc8090.cl · July 2026
Benchmark, harness, and full transcripts: github.com/lnds/kaikai-authorability-bench

Abstract. Large language models write mainstream programming languages from years of training data. A new language has none. We ask whether **structured tooling** — a machine-queryable language reference, typed holes, and diagnostics written to be acted upon — can substitute for that missing prior. We present an authorability benchmark for **kaikai**, a statically typed functional language with zero presence in any training corpus, evaluated head-to-head against Rust and Go across 511 agent-executed task cells: 65 tasks in two size tiers, four LLMs from two providers, and enforced tooling ablations. Three findings. First, correctness saturates: every model — including an open-weight, non-Anthropic one — matches its Rust/Go first-round success rate in kaikai (≈100% throughout; the open-weight model's only failures were in the mainstream languages). Second, the missing prior is paid in **tokens**, not correctness: kaikai costs 4–15× the tokens of a known language at function scale, a tax that scales inversely with model strength and — for the stronger models — amortizes by roughly half when tasks grow to module size, while the open-weight model re-pays it flat on every task. Third, the value of the machine-queryable reference has a **sign that depends on the task**: removing it costs +26% tokens on stdlib-heavy tasks but **saves** 26–58% on algorithm-heavy ones — a "curiosity tax" that weaker models pay most. The benchmark's red rounds, though rare, are precisely the designed failure mode: prior collisions caught by the compiler, explained by a diagnostic, and repaired in one round.

1 Introduction

A language model's fluency in a programming language is, to first order, a function of how much of that language it saw during training. This puts new languages in a cold-start bind: nobody writes them because models can't, and models can't because nobody writes them.

kaikai[1] is a statically typed functional language (HM inference, algebraic effects, fibers) designed with an explicit bet: that the cold-start can be broken with **tooling designed for LLM consumption** — a language reference queryable as JSON (`kai info --json`), typed holes that report expected types and in-scope bindings, and compiler diagnostics that state the repair, not just the error. The language's acceptance criteria encode the bet: *"an LLM with JSON access completes the top 80% of typical functions within one round of compilation."*

This paper reports the benchmark built to test that claim adversarially. The design point is asymmetry, declared up front: the models evaluated carry years of prior for Rust and Go and **zero** for kaikai — no documentation, no tutorials, no Stack Overflow answers exist in any training corpus. The benchmark therefore runs uphill for kaikai by construction. If kaikai matches the incumbents, the thesis is proven in its strongest form; where it loses, per-task transcripts say exactly which diagnostic or affordance failed, feeding the language's DX work queue.

2 Benchmark design

2.1 Protocol

Each **cell** is one task × language × seed. The agent receives the task specification, the exact target signature, and access to the language's standard toolchain inside an empty scratch directory (kaikai:

[1] https://github.com/lnds/kaikai

`kai run`, `kai info --json`, typed holes via `kai build --holes`; Rust: `rustc`; Go: `go build` / `go vet`). It delivers a solution file; the harness assembles it with **hidden tests** the agent never sees, compiles, and runs. On red, the compiler or test output is fed back verbatim and the agent tries again, up to five rounds. Everything is logged: rounds, tokens, model identity, full per-round transcripts.

Metrics: **success@1** (green on the first compile), **rounds-to-green** (compile attempts; agent infra errors such as timeouts are logged separately and do not count as attempts), and **tokens per cell** (total, including the agent's internal exploration turns).

Fairness controls: identical hidden tests adapted per language; the same prompt shape; no task requires O(1) random access as the only reasonable solution, so kaikai's cons-list idiom competes fairly with slice-based Rust and Go; every task ships a reference solution in all three languages, and a task enters the set only when all three references pass their hidden tests (a 195/195 smoke gate at time of writing).

2.2 Tooling ablations, enforced

kaikai runs in four configurations: **full**, **no-info** (reference tooling unavailable), **no-holes**, and **flat-diags** (rich diagnostics reduced to traditional `file:line:col: error: message` one-liners). Ablations are **enforced**, not merely prompt-level: a generated `kai` shim on the agent's PATH rejects the ablated commands and, under flat-diags, filters every diagnostic the agent sees — its own probes and the harness feedback alike.

2.3 Tasks and models

65 tasks: 56 single-function (including an edge-dense "hard" subset: RFC 4648 Base64, ISO-8601 week numbering, strict CSV quoting, RPN evaluation with strict token rules) and 9 multi-function modules balanced across three stdlib-surface classes (algorithm-heavy, record/state-heavy, string/stdlib-heavy). Specifications are fully self-contained; hidden tests may demand nothing the signature does not declare.

Models: Claude Fable 5, Claude Opus 4.8, Claude Sonnet 5 (via the `claude` CLI), and GLM-5.2, an open-weight non-Anthropic model (via the `opencode` CLI) — the latter also validating that the harness protocol generalizes across agent CLIs. 511 valid cells total; cells whose every round died on agent infrastructure (e.g. an API spend-limit incident) are excluded and reported separately.

3 Results

3.1 Correctness saturates — for every model, in every language

model	tier	Go s@1	Rust s@1	kaikai s@1
Fable 5	functions	1.00	1.00	1.00
Fable 5	modules	1.00	1.00	1.00
Opus 4.8	functions	1.00	1.00	1.00
Opus 4.8	modules	1.00	1.00	1.00
Sonnet 5	functions	1.00	1.00	1.00
Sonnet 5	modules	1.00	1.00	0.94
GLM-5.2	functions	1.00	1.00	1.00
GLM-5.2	modules	0.94	0.94	**1.00**

Table 1: First-round success by model, tier, and language (full tooling configuration). Every value at or above 0.94; the module tier is 2 seeds × 9 modules per language.

Across 511 cells, no language ever separated from another on correctness by more than one cell (Table 1). The Tier-3 acceptance claim asked for 80% of functions green in one round; the measured value is ≈100% — and it holds one tier up, at module scale, and across providers. Notably, GLM-5.2 — open-weight, non-Anthropic, with zero kaikai prior — was **perfect in kaikai** (30/30 green@1) while its only two failures landed in Rust and Go, its training-prior languages. Two cells prove nothing alone; the sign, however, is the opposite of the null hypothesis, and it makes the thesis provider-independent rather than an artifact of one vendor's models.

The saturation itself is a finding: edge-dense specifications, exact-output contracts, and round-trip invariants do not dent current frontier models at function or module size. Correctness pressure for future iterations must come from repository-scale tasks or from kaikai-specific surface (effects and handlers) where mainstream idiom offers no crutch.

3.2 The missing prior is paid in tokens

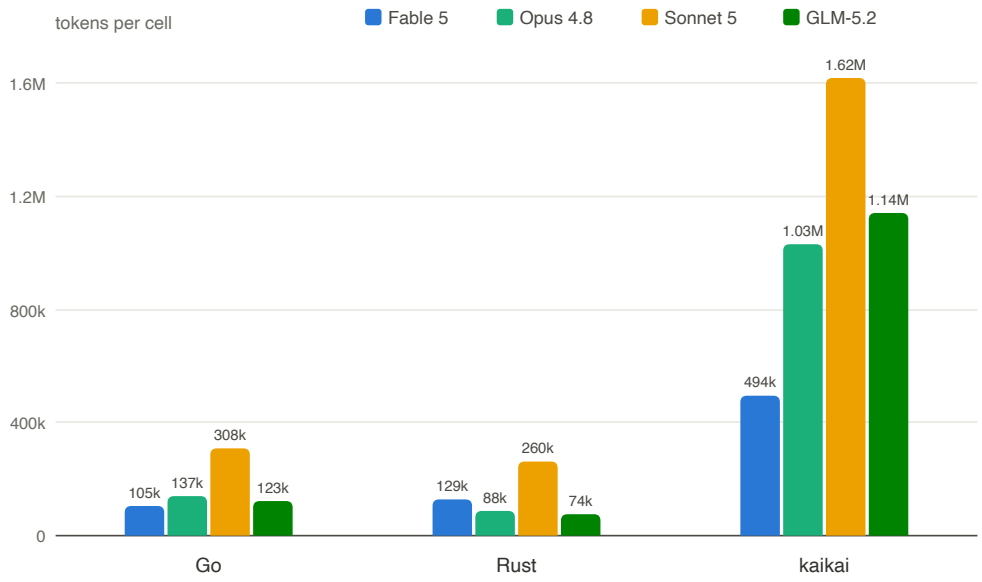


Figure 1: Tokens per cell on the hard-function batch, full configuration. kaikai costs a multiple of that model's cheaper mainstream language, and the multiple grows as the model tier drops: Fable 4.7×, Opus 11.7×, Sonnet 6.2×, GLM-5.2 15.4×.

Correctness parity is not free (Figure 1). At function scale, writing kaikai costs Fable ≈494k tokens per cell against ≈105–129k for Go/Rust; Opus pays ≈1.03M; Sonnet ≈1.62M; GLM-5.2 ≈1.14M against its ≈74–123k mainstream baseline. The missing prior is being purchased at run time — the agent reads the reference, probes with `kai run`, inspects holes — and **weaker models buy more of it**. Token counts include cached context re-reads (the dominant term), so these are utilization figures rather than dollar costs; as a concrete anchor, GLM-5.2's 90 cells cost $59 total at list prices.

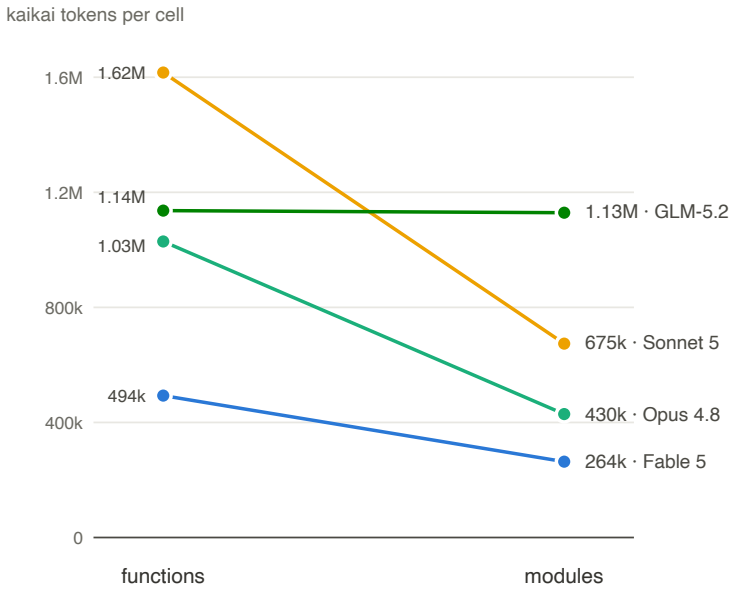


Figure 2: kaikai tokens per cell, functions → modules (full configuration). For the Claude models the exploration cost amortizes as the task grows; GLM-5.2 re-pays the full tax per task, flat across tiers.

For the Claude models the tax also **amortizes** (Figure 2): moving from single functions to multi-function modules — more code per cell, not less — kaikai's per-cell cost **drops** 47–58% (Fable 494k→264k, Opus 1.03M→430k, Sonnet 1.62M→675k), while mainstream languages stay roughly flat. Exploring the language is a fixed cost paid per task; the more code that exploration supports, the cheaper the missing prior becomes per unit of output. GLM-5.2 is the counterexample that sharpens the claim: its kaikai cost is flat across tiers (1.14M→1.13M) — it re-pays the full exploration on every task. Amortization, then, is not a property of the task structure but a **capability of the model**: the stronger models consolidate what they learned about the language within a session and spend it across the larger task; the weaker one explores anew each time. The extrapolation — unverified — is that at repository scale the prior tax approaches the noise floor for models that amortize, and remains a constant multiple for those that do not.

3.3 The reference has task-dependent sign

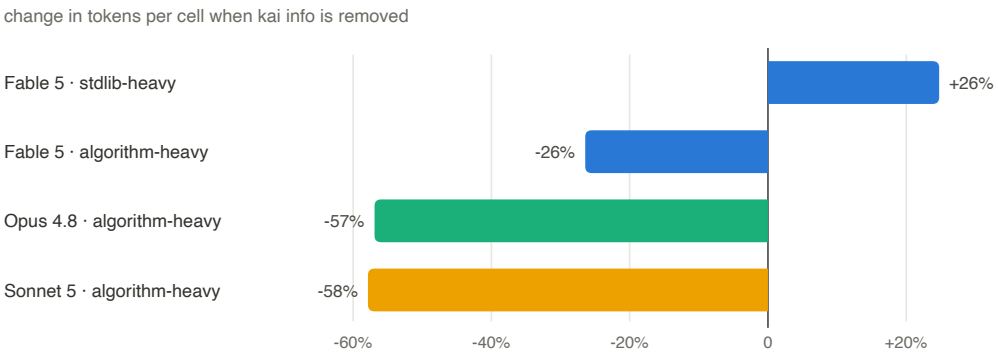


Figure 3: Enforced no-info ablation: change in kaikai tokens per cell when the machine-queryable reference (`kai info`, `kai doc`) is removed, relative to the full configuration on the same tasks.

The ablation grid produced the least intuitive result (Figure 3). On **stdlib-heavy** tasks, removing `kai info` costs +26% tokens (Fable): the agent rediscovers APIs by trial-and-error compilation, exactly as the tooling thesis predicts. But on **algorithm-heavy** tasks — bit packing, calendar arithmetic, hand-rolled parsers that barely touch the standard library — removing the reference made every

model **cheaper**: −26% (Fable), −57% (Opus), −58% (Sonnet). An available reference is an invitation to read it; weaker models accept that invitation far beyond need. We call this the **curiosity tax**.

Two design consequences follow. For benchmark construction: the no-info coefficient is not a constant, so a task set must balance and tag both classes. For language design: the reference should be **cheap to consult at symbol granularity** rather than page granularity — this finding was filed against kaikai's tooling as a concrete issue (symbol-sized answers as the documented default, a ≈20-line prior-collision cheat card, TL;DR discipline on topic pages)[2].

The other two ablations (no-holes, flat-diags) sat within noise (±6%) — with near-zero red rounds there is almost no feedback channel for rich diagnostics to improve, and single-function bodies give typed holes nothing to decompose. Their coefficients await tasks that actually fail.

3.4 What the red rounds contain

Among 511 cells, exactly four reds were genuine model failures — and their content is the benchmark's qualitative payload:

1. **Syntax-level prior collision.** Opus, in kaikai under no-info, wrote `//` comments (Rust/Go habit). The diagnostic answered *"kaikai uses `#` for comments, not `//` (and `//` is not an operator — `Int / Int` already truncates)"* — green next round. The collision surfaced only with the reference removed: the tooling layers back each other up.
2. **Idiom-level prior collision.** Sonnet, in kaikai, transplanted the imperative accumulate-and-reverse pattern onto cons lists and corrupted element order — a clean compile, a failed hidden assertion, and a recovery (from a bare "FAIL") by rewriting into idiomatic structural recursion.
3. & 4. **Logic misses in mainstream languages.** GLM-5.2′s two failures: an `Option::unwrap` panic in Rust and a failed assertion in Go. Both green next round.

Every red recovered in exactly one round. The loop the language was designed around — prior collision → caught by compiler → explained by diagnostic → repaired — is observable end to end, just far rarer than anticipated.

4 Discussion

The benchmark set out to measure whether structured tooling compensates for a missing training prior. The answer has a shape we did not fully anticipate. On the **correctness** axis the compensation is total — indistinguishable from parity across two task tiers, four models, and two providers — which means the axis itself stops discriminating. What remains, and what discriminates in every experiment, is the **cost** axis: how many tokens a model spends conversing with the toolchain to reach the same green. That reframing matters for language designers: "designed for LLMs" is not one coefficient but a family — cost by model tier, by task's stdlib surface, by task size — and at least one member (the curiosity tax) actively rewards making documentation **smaller** at the point of consumption.

The result also suggests a practical adoption argument for new languages generally: the cold-start problem is real but it is a **pricing** problem, not a capability problem, and it shrinks with task size. A team choosing kaikai for module-scale work pays roughly 2–3× in agent tokens today, not the 5–12× the function-scale numbers suggest, and none of it in correctness.

5 Threats to validity

Small samples per cell. Most grids run 2 seeds per cell (calibration used 5); the per-model coefficients carry meaningful variance, and single-red observations (Opus's 0.92, Sonnet's 0.94) are anecdotes with the right sign, not measurements. **One task author.** All 65 tasks were authored

[2] https://github.com/lnds/kaikai/issues/1166

within this project; a pilot run caught one unfair hidden test (requiring undeclared trait derives in Rust), which was fixed and its cells re-run — other biases may remain. **Token counts are utilization, not cost**, dominated by cached re-reads; GLM-5.2′s counts were recovered post-hoc from its CLI's local session store rather than captured by the harness, an asymmetry in provenance though not in definition. **Saturation bounds the instrument.** With success@1 ≈ 1.0 everywhere, correctness comparisons are uninformative at these tiers for these models; claims are correspondingly about cost. **The asymmetry ages.** kaikai's absence from training corpora is what makes the experiment clean; the moment kaikai code (including this benchmark) enters a training set, the uphill condition erodes.

6 Related work

Code-generation benchmarks in the HumanEval/MBPP tradition measure correctness of single-shot completions in languages the models know well; SWE-bench moved the target to repository-scale edits, still within high-prior ecosystems. This benchmark differs on the independent variable: it holds the tasks fixed across languages and varies the **training prior and its tooling substitute**, using agents with compiler access rather than one-shot generation, and treating tokens-to-green as a first-class outcome rather than an implementation detail.

7 Conclusion

With structured tooling, having never existed in a training corpus turned out to be a cost problem, not a correctness problem. The cost has structure — it scales inversely with model strength, amortizes with task size, and one of its components (reference consultation) changes sign with the task's library surface. The benchmark, its 65 tasks, the enforced ablation harness, and every transcript are public and re-runnable; each future kaikai release runs it again, and each red round becomes a language issue. The ratchet is the point.

Acknowledgments. Experiment orchestration, harness implementation, and task authoring were done in collaboration with Claude Code (Anthropic). GLM-5.2 was driven via the open-source `opencode` CLI.

References. Chen et al., *Evaluating Large Language Models Trained on Code* (HumanEval), arXiv:2107.03374, 2021. · Austin et al., *Program Synthesis with Large Language Models* (MBPP), arXiv:2108.07732, 2021. · Jimenez et al., *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?*, arXiv:2310.06770, 2023. · kaikai: github.com/lnds/kaikai. · This benchmark: github.com/lnds/kaikai-authorability-bench.